

RELATÓRIO TÉCNICO: ANÁLISE EXPERIMENTAL DE ESTRUTURAS DE DADOS E ORDENAÇÃO

Disciplina: Estruturas, Pesquisa e Ordenação de Dados

Professor: MSc. Gabriel Passos de Jesus

Autores: Alexandre Gustavo Mota Ribeiro, Giovane Canteri da Silva, Higor Assmé Gondin, Matheus Iuri dos Santos Tórtura, Matheus Ramos da Silva.

Data: 09/04/2026

INTRODUÇÃO

Este relatório técnico apresenta a análise empírica e teórica de algoritmos clássicos de estruturação, pesquisa e ordenação de dados. O projeto foi dividido em três frentes de experimentação: (I) Árvores e Balanceamento e o Problema do Caixeiro-Viajante, (II) Sistemas de Busca e (III) Benchmark de Ordenação. O objetivo principal é entender a complexidade assintótica teórica dos algoritmos com o seu desempenho real em hardware, observando como o tempo de execução e o esforço computacional (comparações e movimentações) se comportam sob diferentes volumes e disposições de dados.

FUNDAMENTAÇÃO TEÓRICA

Uma árvore binária de busca (BST) é uma estrutura hierárquica não linear que organiza dados a partir de uma raiz. Apesar de eficiente, a BST simples pode sofrer degeneração estrutural dependendo da ordem de inserção dos dados. Para suavizar esse problema, implementou-se a árvore AVL, que aplica um controle de balanceamento onde a diferença de altura entre as subárvores esquerda e direita deve ser menor ou igual a 1 para todo nó, utilizando rotações estruturais caso haja violação dessa regra. Paralelamente, a árvore Rubro-Negra flexibiliza esse balanceamento contínuo através de propriedades de cor, como a exigência de que a raiz seja preta e a restrição de que um nó vermelho não pode possuir filhos vermelhos.

No contexto dos sistemas de busca, a Busca Sequencial realiza uma varredura linear que se torna ineficiente para grandes volumes de dados. Em contrapartida, a Busca Binária aplica o princípio de divisão e conquista em conjuntos previamente ordenados, reduzindo drasticamente o espaço de busca. Para a ordenação desses conjuntos, algoritmos clássicos possuem propriedades importantes, como estabilidade e execução in-place. O Merge Sort utiliza uma abordagem recursiva de divisão e mesclagem que garante a estabilidade dos dados. Já o Quick Sort particiona o array com base em um pivô que, nesta implementação experimental, foi escolhido de forma aleatória com o intuito de otimizar o desempenho prático e evitar o pior caso computacional.

ANÁLISE ASSINTÓTICA

A notação Big-O é usada para descrever o limite do crescimento de uma função em relação ao tamanho de sua entrada. Matematicamente, se uma árvore binária é perfeitamente balanceada e possui n nós, demonstra-se que a sua altura satisfaz um limite assintótico $\Theta(\log n)$. Isso garante que as operações de busca, inserção e remoção nas árvores AVL e Rubro-Negra operem sempre em tempo logarítmico $O(\log n)$, sendo que a Rubro-Negra possui uma altura máxima proporcional a duas vezes o logaritmo natural da quantidade de nós ($\leq 2 \log(n+1)$). Em contrapartida, em seu pior cenário estrutural, a árvore BST simples sofre degeneração e passa a operar de forma semelhante a uma lista encadeada.

Em relação aos métodos, a Busca Sequencial possui um limite superior linear $O(n)$, enquanto a Busca Binária opera eficientemente em tempo logarítmico $O(\log n)$. No benchmark de ordenação, o algoritmo Merge Sort garante uma complexidade de tempo log-linear $O(n \log n)$ constante em todos os cenários possíveis. O Quick Sort, embora possua um limite de pior caso quadrático $O(n^2)$, alcança uma média log-linear altamente eficiente e escalável com a estratégia de pivô aleatório aplicada nesta análise.

METODOLOGIA EXPERIMENTAL

Para garantir a integridade estatística, todos os algoritmos foram submetidos ao mesmo ambiente de testes.

- **Bateria de Testes:** Cada cenário foi executado 30 vezes. Os valores apresentados neste relatório representam a média dessas execuções, acompanhados de seu desvio padrão quando aplicável.
- **Métricas Coletadas:** Tempo de execução (medido em nanossegundos via `System.nanoTime()`), quantidade de comparações e quantidade de movimentações de dados (atribuições, trocas e rotações).
- **Cenários Estressados:** Avaliação do Melhor Caso, Caso Médio (com dados aleatórios gerados pela seed 42 para reprodutibilidade) e Pior Caso (dados ordenados ou invertidos, de acordo com o algoritmo).
- **Tamanhos de Entrada:** Variaram tipicamente entre 1.000, 10.000 e 100.000 elementos, ajustando-se para proporções menores no Caixeiro-Viajante (10, 50, 100).

RESULTADOS

Todos os resultados a seguir foram obtidos com geração aleatória de dados.

Projeto 1 - Árvores e Balanceamento

N = 100

```
Tamanho n = 100
-----
Algoritmo: BST                               Tamanho: 100
Media: 29137 ns   Desvio padrao: 42464 ns

Algoritmo: AVL                               Tamanho: 100
Media: 85837 ns   Desvio padrao: 86233 ns

Algoritmo: Rubro-Negra (LLRB)                Tamanho: 100
Media: 94993 ns   Desvio padrao: 107811 ns
```

N = 1.000

```
Tamanho n = 1000
-----
Algoritmo: BST                               Tamanho: 1000
Media: 149840 ns   Desvio padrao: 103912 ns

Algoritmo: AVL                               Tamanho: 1000
Media: 275353 ns   Desvio padrao: 100580 ns

Algoritmo: Rubro-Negra (LLRB)                Tamanho: 1000
Media: 230590 ns   Desvio padrao: 220746 ns
```

N = 10.000

```
Tamanho n = 10000
-----
Algoritmo: BST                               Tamanho: 10000
Media: 1201630 ns   Desvio padrao: 228290 ns

Algoritmo: AVL                               Tamanho: 10000
Media: 1323517 ns   Desvio padrao: 448831 ns

Algoritmo: Rubro-Negra (LLRB)                Tamanho: 10000
Media: 1742567 ns   Desvio padrao: 272485 ns
```

Projeto 2 - Sistemas de Busca

$N = 1.000$

Tamanho n = 1000

Algoritmo: Sequencial Tamanho: 1000
Media: 27207 ns Desvio padrao: 111652 ns

Algoritmo: Binaria Tamanho: 1000
Media: 17317 ns Desvio padrao: 92638 ns

Algoritmo: Busca BST Tamanho: 1000
Media: 497 ns Desvio padrao: 488 ns

$N = 10.000$

Tamanho n = 10000

Algoritmo: Sequencial Tamanho: 10000
Media: 29157 ns Desvio padrao: 27567 ns

Algoritmo: Binaria Tamanho: 10000
Media: 837 ns Desvio padrao: 448 ns

Algoritmo: Busca BST Tamanho: 10000
Media: 1070 ns Desvio padrao: 1702 ns

$N = 100.000$

Tamanho n = 100000

Algoritmo: Sequencial Tamanho: 100000
Media: 12647 ns Desvio padrao: 19087 ns

Algoritmo: Binaria Tamanho: 100000
Media: 983 ns Desvio padrao: 1073 ns

Algoritmo: Busca BST Tamanho: 100000
Media: 437 ns Desvio padrao: 483 ns

Projeto 3 - Benchmark de Ordenação

1. Merge Sort

$N = 1.000$

```
Tamanho n = 1000
-----
Algoritmo: MergeSort - Melhor caso Tamanho: 1000
Media: 363413 ns   Desvio padrao: 1632838 ns

Algoritmo: MergeSort - Caso medio Tamanho: 1000
Media: 132183 ns   Desvio padrao: 75700 ns

Algoritmo: MergeSort - Pior caso Tamanho: 1000
Media: 93917 ns   Desvio padrao: 22837 ns
```

$N = 10.000$

```
Tamanho n = 10000
-----
Algoritmo: MergeSort - Melhor caso Tamanho: 10000
Media: 530623 ns   Desvio padrao: 293503 ns

Algoritmo: MergeSort - Caso medio Tamanho: 10000
Media: 738300 ns   Desvio padrao: 60050 ns

Algoritmo: MergeSort - Pior caso Tamanho: 10000
Media: 316560 ns   Desvio padrao: 75782 ns
```

$N = 100.000$

```
Tamanho n = 100000
-----
Algoritmo: MergeSort - Melhor caso Tamanho: 100000
Media: 2990223 ns   Desvio padrao: 600037 ns

Algoritmo: MergeSort - Caso medio Tamanho: 100000
Media: 8930257 ns   Desvio padrao: 358831 ns

Algoritmo: MergeSort - Pior caso Tamanho: 100000
Media: 2823320 ns   Desvio padrao: 336776 ns
```

2. Quick Sort

$N = 1.000$

```
Tamanho n = 1000
-----
Algoritmo: QuickSort - Melhor caso Tamanho: 1000
Media: 302927 ns   Desvio padrao: 1320639 ns

Algoritmo: QuickSort - Caso medio Tamanho: 1000
Media: 58447 ns   Desvio padrao: 15958 ns

Algoritmo: QuickSort - Pior caso Tamanho: 1000
Media: 27733 ns   Desvio padrao: 1198 ns
```

$N = 10.000$

```
Tamanho n = 10000
-----
Algoritmo: QuickSort - Melhor caso Tamanho: 10000
Media: 278183 ns   Desvio padrao: 62173 ns

Algoritmo: QuickSort - Caso medio Tamanho: 10000
Media: 608783 ns   Desvio padrao: 74643 ns

Algoritmo: QuickSort - Pior caso Tamanho: 10000
Media: 289010 ns   Desvio padrao: 21300 ns
```

$N = 100.000$

```
Tamanho n = 100000
-----
Algoritmo: QuickSort - Melhor caso Tamanho: 100000
Media: 2851013 ns   Desvio padrao: 160625 ns

Algoritmo: QuickSort - Caso medio Tamanho: 100000
Media: 6874143 ns   Desvio padrao: 189062 ns

Algoritmo: QuickSort - Pior caso Tamanho: 100000
Media: 3101737 ns   Desvio padrao: 165268 ns
```

Caixeiro-viajante – Vizinho mais próximo(heurística)

```
Numero de cidades = 10
```

```
-----  
Algoritmo: Caixeiro Viajante      Tamanho: 10  
Media: 3413 ns   Desvio padrao: 2390 ns
```

```
(custo da 1a execucao: 187)
```

```
Numero de cidades = 50
```

```
-----  
Algoritmo: Caixeiro Viajante      Tamanho: 50  
Media: 41520 ns   Desvio padrao: 8664 ns
```

```
(custo da 1a execucao: 313)
```

```
Numero de cidades = 100
```

```
-----  
Algoritmo: Caixeiro Viajante      Tamanho: 100  
Media: 56117 ns   Desvio padrao: 16848 ns
```

```
(custo da 1a execucao: 475)
```

DISCUSSÃO

No **Projeto 1**, a árvore BST apresentou o menor tempo de execução com dados aleatórios, mas, em outro teste, sofreu falha sistêmica (*StackOverflowError*) ao processar 100.000 elementos ordenados. Sem mecanismos de balanceamento, a estrutura degenerou para uma formação linear $O(n)$, excedendo o limite de chamadas recursivas da linguagem. Em contrapartida, as árvores AVL e Rubro-Negra (LLRB) mantiveram a eficiência logarítmica $O(\log n)$ e processaram a carga extrema com estabilidade.

No **Projeto 2**, ficou evidente a diferença de escalabilidade entre os métodos de pesquisa. A Busca Sequencial demonstrou alto custo computacional ao varrer vetores de grande porte $O(n)$. Já a Busca Binária localizou os mesmos valores numa fração de nanosegundos, validando a expressiva eficiência da abordagem de divisão e conquista $O(\log n)$ em conjuntos previamente ordenados.

No **Projeto 3**, o algoritmo Quick Sort apresentou um comportamento empírico notável. Teoricamente, vetores invertidos deveriam forçar a sua degradação para o pior caso $O(n^2)$. No entanto, a estratégia de utilizar um **pivô aleatório** mitigou essa vulnerabilidade matemática. Além disso, os padrões lógicos dos vetores estruturados (ordenados e invertidos) otimizaram o uso de recursos de *hardware* do processador (como a previsão de desvios - *Branch Prediction*), resultando em tempos de execução ainda menores do que os observados no cenário aleatório.

No problema do Caixeiro Viajante, a heurística do Vizinho Mais Próximo substituiu a solução exata inviável $O(n!)$ por um modelo muito mais rápido de $O(n^2)$. Isso ilustra o clássico *trade-off* computacional: sacrifica-se a garantia da rota perfeita em troca de um tempo de execução viável e escalável.

Em síntese, os experimentos evidenciam que o desempenho de tempo e memória de um algoritmo depende fortemente da organização inicial da sua entrada. A falha estrutural da BST e a resiliência do Quick Sort comprovam que a escolha de uma solução de *software* não deve focar apenas no seu "Caso Médio" teórico, mas sim prever e suportar o comportamento dos dados extremos no ambiente real de operação.

CONCLUSÃO

A experimentação prática validou as premissas matemáticas do estudo de complexidade. A implementação direta dos algoritmos e das estruturas auto balanceadas demonstrou que a escolha da estrutura de dados correta é crítica não apenas em termos de processamento de CPU (comparações), mas também na gestão de memória (movimentações e alocações auxiliares). O desenvolvimento da suíte de validação confirmou a integridade das operações e permitiu uma entrega de software robusta e alinhada às exigências do mercado.